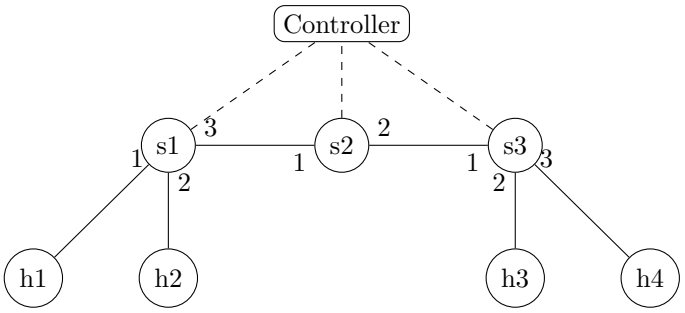
# Piattaforme Software per la Rete (Mod. 1) – 30/06/2015

Su ogni foglio consegnato scrivere nome, cognome, matricola e il numero dell'esercizio.

**Esercizio 1**

Si consideri la rete in figura. I nodi h1,..., h$n$ contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., h$n$ sono MAC1,..., MAC$n$ e IP1,..., IP$n$. Sui nodi h1,..., h$n$ sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse.




Il controllore Ryu è collegato ai nodi s1,..., s$m$ ed è configurato come indicato nel Programma 1.

Scrivere tutti i pacchetti che sono inviati su tutti i collegamenti della rete e tra i nodi e il controllore in coseguenza degli eventi seguenti. Scrivere il contenuto delle tabelle openflow prima e dopo ogni domanda.

**Programma 1 (1/2)**

```
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet, ethernet, ipv4, tcp

class PsrSwitch(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                          ofproto.OFPCML_NO_BUFFER)]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

        if datapath.id == 2: # switch s2
            match = parser.OFPMatch(eth_type=0x0800, # IPv4
                                    ip_proto=17, # UDP
                                    udp_dst=53)
            actions = [
                parser.OFPActionSetField(eth_dst=MAC2),
                parser.OFPActionSetField(ip_dst=IP2),
                parser.OFPActionOutput(1)
                      ]
            inst = [parser.OFPInstructionActions(
                   ofproto.OFPIT_APPLY_ACTIONS, actions)]
            mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                    match=match, instructions=inst)
            datapath.send_msg(mod)
```

**Programma 1 (2/2)**

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    msg = ev.msg
    datapath = msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    in_port = msg.match['in_port']

    pkt = packet.Packet(msg.data)
    pkt_eth = pkt.get_protocol(ethernet.ethernet)
    pkt_ip = pkt.get_protocol(ipv4.ipv4)
    pkt_tcp = pkt.get_protocol(tcp.tcp)

    if ((datapath.id == 1) or (datapath.id == 3))
       and (pkt_tcp is not None):
        if pkt_eht.dst == MAC4:
            out_port = 3
        else:
            out_port = 1

        match = parser.OFPMatch(
            eth_type=0x0800,
            ipv4_src=pkt_ip.src,ipv4_dst=pkt_ip.dst,ip_proto=6,
            tcp_src=pkt_tcp.src_port,tcp_dst=pkt_tcp.dst_port
        )

        actions = [parser.OFPActionOutput(out_port)]

        inst = [parser.OFPInstructionActions(
               ofproto.OFPIT_APPLY_ACTIONS, actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=2,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
        return

    data = msg.data
    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath,
              buffer_id=msg.buffer_id,in_port=in_port,
              actions=actions, data=data)
    datapath.send_msg(out)
```

(a) Invio di un nuovo datagramma UDP da h1:5432 a h2:53 e relativa risposta.

**Soluzione:**

stato iniziale delle tabelle

s1

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 0 |

s2

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 0 |
| 1 | eth_type=ipv4, ip_proto=udp, udp_dst=53 | set eth_dst=MAC2, set ip_dst=IP2, output port 1 | 0 |

s3

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 0 |

Invio da h1 a h2

| | |
|---|---|
| $h1 \rightarrow s1$ | MAC1 MAC2 IPv4 IP1 IP2 UDP 5432 53 |
| $s1 \rightarrow C$ | packet_in(. . . ) |
| $C \rightarrow s1$ | packet_out(. . . ,output=FLOOD) |
| $s1 \rightarrow h2$ | . . . |
| $s1 \rightarrow s2$ | . . . |
| $s2 \rightarrow s1$ | . . . |
| $s1 \rightarrow C$ | packet_in(. . . ) |
| $C \rightarrow s1$ | packet_out(. . . ,output=FLOOD) |
| $s1 \rightarrow h1$ | . . . |
| $s1 \rightarrow h2$ | . . . |

h2 riceve due copie del pacchetto e invia due risposte
prima risposta

| | |
|---|---|
| $h2 \rightarrow s1$ | MAC2 MAC1 IPv4 IP2 IP1 UDP 53 5432 |
| $s1 \rightarrow C$ | packet_in(. . . ) |
| $C \rightarrow s1$ | packet_out(. . . ,output=FLOOD) |
| $s1 \rightarrow h1$ | . . . |
| $s1 \rightarrow s2$ | . . . |
| $s2 \rightarrow C$ | packet_in(. . . ) |
| $C \rightarrow s2$ | packet_out(. . . ,output=FLOOD) |
| $s2 \rightarrow s3$ | . . . |
| $s3 \rightarrow C$ | packet_in(. . . ) |
| $C \rightarrow s3$ | packet_out(. . . ,output=FLOOD) |
| $s3 \rightarrow h3$ | . . . |
| $s3 \rightarrow h4$ | . . . |

la seconda risposta è identica
stato finale delle tabelle

s1

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 4 |

s2

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 2 |
| 1 | eth_type=ipv4, ip_proto=udp, udp_dst=53 | set eth_dst=MAC2, set ip_dst=IP2, output port 1 | 1 |

s3

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 2 |

(b) Invio di un datagramma UDP da h2:5432 a h4:53 e relativa risposta;

**Soluzione:**
Invio da h2 a h4

| | |
|---|---|
| $h1 \rightarrow s1$ | MAC2 MAC4 IPv4 IP2 IP4 UDP 5432 53 |
| $s1 \rightarrow C$ | packet_in(. . . ) |
| $C \rightarrow s1$ | packet_out(. . . ,output=FLOOD) |
| $s1 \rightarrow h1$ | . . . |
| $s1 \rightarrow s2$ | . . . |
| $s2 \rightarrow s1$ | MAC2 MAC2 IPv4 IP2 IP2 UDP 5432 53 |
| $s1 \rightarrow C$ | packet_in(. . . ) |
| $C \rightarrow s1$ | packet_out(. . . ,output=FLOOD) |
| $s1 \rightarrow h1$ | . . . |
| $s1 \rightarrow h2$ | . . . |

h2 risponde, ma il pacchetto rimane locale all'host
stato finale delle tabelle

s1

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 6 |

s2

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 2 |
| 1 | eth_type=ipv4, ip_proto=udp, udp_dst=53 | set eth_dst=MAC2, set ip_dst=IP2, output port 1 | 2 |

s3

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 2 |

(c) Apertura di una connessione TCP da h1:12345 a h4:80;

**Soluzione:**
Invio da h1 a h4

| | |
|---|---|
| $h1 \rightarrow s1$ | MAC1 MAC4 IPv4 IP1 IP4 TCP 12345 80 |
| $s1 \rightarrow C$ | packet_in(...) |
| $C \rightarrow s1$ | flow_mod(priority=2, match=( eth_type=IPv4, ip_src=IP1, ip_dst=IP4, ip_proto=TCP, tcp_src=12345, tcp_dst=80), action=ouput(3) ) |

scade il timeout e h1 ritrasmette

| | |
|---|---|
| $h1 \rightarrow s1$ | MAC1 MAC4 IPv4 IP1 IP4 TCP 12345 80 |
| $s1 \rightarrow s2$ | ... |
| $s2 \rightarrow C$ | packet_in(...) |
| $C \rightarrow s2$ | packet_out(...,output=FLOOD) |
| $s2 \rightarrow s3$ | ... |
| $s3 \rightarrow C$ | packet_in(...) |
| $C \rightarrow s3$ | flow_mod(priority=2, match=( eth_type=IPv4, ip_src=IP1, ip_dst=IP4, ip_proto=TCP, tcp_src=12345, tcp_dst=80), action=ouput(3) ) |

scade il timeout e h1 ritrasmette

| | |
|---|---|
| $h1 \rightarrow s1$ | MAC1 MAC4 IPv4 IP1 IP4 TCP 12345 80 |
| $s1 \rightarrow s2$ | ... |
| $s2 \rightarrow C$ | packet_in(...) |
| $C \rightarrow s2$ | packet_out(...,output=FLOOD) |
| $s2 \rightarrow s3$ | ... |
| $s3 \rightarrow h4$ | ... |

h4 risponde con un SYN ACK

| | |
|---|---|
| $h4 \rightarrow s3$ | MAC4 MAC1 IPv4 IP4 IP1 TCP 80 12345 |
| $s3 \rightarrow C$ | packet_in(...) |
| $C \rightarrow s3$ | flow_mod(priority=2, match=( eth_type=IPv4, ip_src=IP4, ip_dst=IP1, ip_proto=TCP, tcp_src=80, tcp_dst=12345), action=ouput(1) ) |

scade il timeout e h4 ritrasmette

| | |
|---|---|
| $h4 \rightarrow s3$ | MAC4 MAC1 IPv4 IP4 IP1 TCP 80 12345 |
| $s3 \rightarrow s2$ | ... |
| $s2 \rightarrow C$ | packet_in(...) |
| $C \rightarrow s2$ | packet_out(...,output=FLOOD) |
| $s2 \rightarrow s1$ | ... |
| $s1 \rightarrow C$ | packet_in(...) |
| $C \rightarrow s1$ | flow_mod(priority=2, match=( eth_type=IPv4, ip_src=IP4, ip_dst=IP1, ip_proto=TCP, tcp_src=80, tcp_dst=12345), action=ouput(1) ) |

scade il timeout e h4 ritrasmette

| | |
|---|---|
| $h4 \rightarrow s3$ | MAC4 MAC1 IPv4 IP4 IP1 TCP 80 12345 |
| $s3 \rightarrow s2$ | ... |
| $s2 \rightarrow C$ | packet_in(...) |
| $C \rightarrow s2$ | packet_out(...,output=FLOOD) |
| $s2 \rightarrow s1$ | ... |
| $s1 \rightarrow h1$ | ... |

ACK finale

| | |
|---|---|
| $h1 \rightarrow s1$ | MAC1 MAC4 IPv4 IP1 IP4 TCP 12345 80 |
| $s1 \rightarrow s2$ | ... |
| $s2 \rightarrow C$ | packet_in(...) |
| $C \rightarrow s2$ | packet_out(...,output=FLOOD) |
| $s2 \rightarrow s3$ | ... |
| $s3 \rightarrow h4$ | ... |

stato finale delle tabelle

s1

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 7 |
| 2 | eth_type=IPv4, ip_src=IP1, ip_dst=IP4, ip_proto=TCP, tcp_src=12345, tcp_dst=80 | output port 3 | 3 |
| 2 | eth_type=IPv4, ip_src=IP4, ip_dst=IP1, ip_proto=TCP, tcp_src=80, tcp_dst=12345 | output port 1 | 2 |

s2

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 7 |
| 1 | eth_type=ipv4, ip_proto=udp, udp_dst=53 | set eth_dst=MAC2, set ip_dst=IP2, output port 1 | 2 |

s3

| Priority | Match | Action | pkt cnt |
|---|---|---|---|
| 0 | * | output controller | 4 |
| 2 | eth_type=IPv4, ip_src=IP1, ip_dst=IP4, ip_proto=TCP, tcp_src=12345, tcp_dst=80 | output port 3 | 3 |
| 2 | eth_type=IPv4, ip_src=IP4, ip_dst=IP1, ip_proto=TCP, tcp_src=80, tcp_dst=12345 | output port 1 | 1 |

**Esercizio 2**

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

| Source Address | Destination Address | Source Port | Destination Port | Protocol |
|---|---|---|---|---|
| * | E | * | 80 | TCP |
| A | F | 25 | * | UDP |
| B | * | 21 | 22 | TCP |
| C | G | 80 | * | * |
| * | * | 53 | 80 | UDP |
| D | E | * | 123 | * |
| B | H | 80 | * | UDP |
| C | * | 21 | * | TCP |

(a) Definire il bitvector set per ciascun campo.

**Soluzione:**

| Source Address | | Dest. Address | | Source Port | | Dest. Port | | Protocol | |
|---|---|---|---|---|---|---|---|---|---|
| A | 11001000 | E | 10101101 | 21 | 10100101 | 22 | 01110011 | TCP | 10110101 |
| B | 10101010 | F | 01101001 | 25 | 11000100 | 80 | 11011011 | UDP | 01011110 |
| C | 10011001 | G | 00111001 | 53 | 10001100 | 123 | 01010111 | * | 00010100 |
| D | 10001100 | H | 00101011 | 80 | 10010110 | * | 01010011 | | |
| * | 10001000 | * | 00101001 | * | 10000100 | | | | |

(b) Assumendo che gli indirizzi sorgente/destinazione siano codificati su 32 bit, i numeri di porta sorgente/destinazione su 16 bit e il tipo di protocollo su 2 bit, calcolare la dimensione totale (in bit) del bitvector set.

**Soluzione:** (32+8)*10+(16+8)*9+(2+8)*3= 646 bit

(c) Si considerino i pacchetti (B, E, 23, 53, UDP) e (C, E, 21, 123, TCP). Applicare ad entrambi la procedura di BVLS per identificare gli eventuali matching.

**Soluzione:** 1) 10101010 AND 10101101 AND 10000100 AND 01010011 AND 01011110 = 00000000 Nessuna regola soddisfatta. 2) 10011001 AND 10101101 AND 10100101 AND 01010111 AND 10110101 = 00000001 Il pacchetto soddisfa la regola 8.

(d) Spiegare brevemente come può essere implementata la ricerca della parola binaria nei campi Source Address e Protocol.

**Esercizio 3**

Si consideri un sistema di misura del traffico in un router. Ai pacchetti entranti da tutti i link è associato un identificativo di flusso $x$ ottenuto concatenando alcuni campi del pacchetto. La lunghezza dei pacchetti è costante di 512 byte.

Sull'identificativo $x$ sono calcolati $n$ hash usando la famiglia di funzioni indipendenti $h_n(x)$. Tali hash sono usati come indici in $n$ array $A_1, \ldots, A_n$ di $m$ elementi ciascuno. Ad ogni pacchetto arrivato si incrementa di uno la $h_n(x)$-esima posizione di ognuno degli $n$ array.

Se $A_n[h_n(x)] \geq S$ per ogni $n$ il valore $x$ è salvato su disco.

Tutti gli array sono azzerati ogni $T = 10\,\text{s}$.

(a) Dire quali campi occorre scegliere per rappresentare l'identificativo del flusso per avere nello stesso flusso tutti i pacchetti diretti allo stesso socket sullo stesso host.

**Soluzione:**

$$x = \text{IPd} \| \text{proto} \| \text{portd}$$

(b) Considerando un link unidirezionale a $C = 40\,\text{Gbit/s}$, dire quanti bit occorrono per rappresentare gli $n$ array.

**Soluzione:** Nel caso pessimo, in 10 s si accumulano $400 \cdot 10^9$ bit in un unico contatore. Contando in pacchetti servono

$$\lceil \log_2 \frac{400 \cdot 10^9}{8 \times 512} \rceil mn = 27mn \text{ bit}$$

(oppure $39mn$ bit o $36mn$ ottetti)

(c) Dire come scegliere la soglia $S$ in modo che si scriva su disco tutti i flussi da almeno 25 Mbit/s.

**Soluzione:** In dieci secondi il flusso da monitorare accumula almeno 250 Mbit, ovvero $S = 61036$ pacchetti. (oppure $31 \cdot 10^7$ ottetti)

(d) Un certo flusso $y$ ha superato la soglia nell'attuale intervallo di misura. Considerando $m = 2^8$ e $n = 3$, dire qual è la probabilità che un nuovo flusso sia copiato su disco. *Ignorare gli eventuali altri flussi nel sistema.*

**Soluzione:**

$$\left(\frac{1}{m}\right)^n = \frac{1}{2^{24}} \simeq 6 \cdot 10^{-8}$$

(e) Tre flussi $x_1, x_2, x_3$ hanno accumulato $S/2$ pacchetti alla fine dell'intervallo di misura. Considerando $m = 2^8$ e $n = 3$, dire qual è la probabilità che almeno uno di essi sia stato salvato su disco. *Ignorare gli eventuali altri flussi nel sistema.*

**Soluzione:**
Un flusso viene salvato su disco se esso è coinvolto in una collisione in tutte le funzioni hash.
Possiamo dare una valutazione approssimata per eccesso semplicemente valutando la probabilità che ci sia una collisione in tutte le funzioni hash:

$$\left(\frac{3 \times 2}{2m}\right)^n = \frac{27}{2^{27}} \simeq 2 \cdot 10^{-7}$$